

Classification_of_Fashion_accessories

December 30, 2025

Goal: To analyze and Fashion-MNIST data and create a predictive model using deeplearning. Understand the model complexity influences classification accuracy and efficiency on the dataset.

1.1) Library Imports Importing the essential libraries:

- **NumPy** for numerical operations
- **TensorFlow / Keras** for building and training deep learning models
- **Matplotlib** for visualizations

Importing the libraries

```
import numpy as np
import pandas as pd
import tensorflow as tf
from tensorflow import keras
import matplotlib.pyplot as plt

import plotly.graph_objects as go
from plotly.subplots import make_subplots
```

1.2) Load Dataset Load the **Fashion-MNIST** dataset from Keras, splitting it into:

- `train_images, train_labels` → training set
- `test_images, test_labels` → test set

```
(train_images, train_labels), (test_image, test_labels) = keras.datasets.fashion_mnist.load_data
```

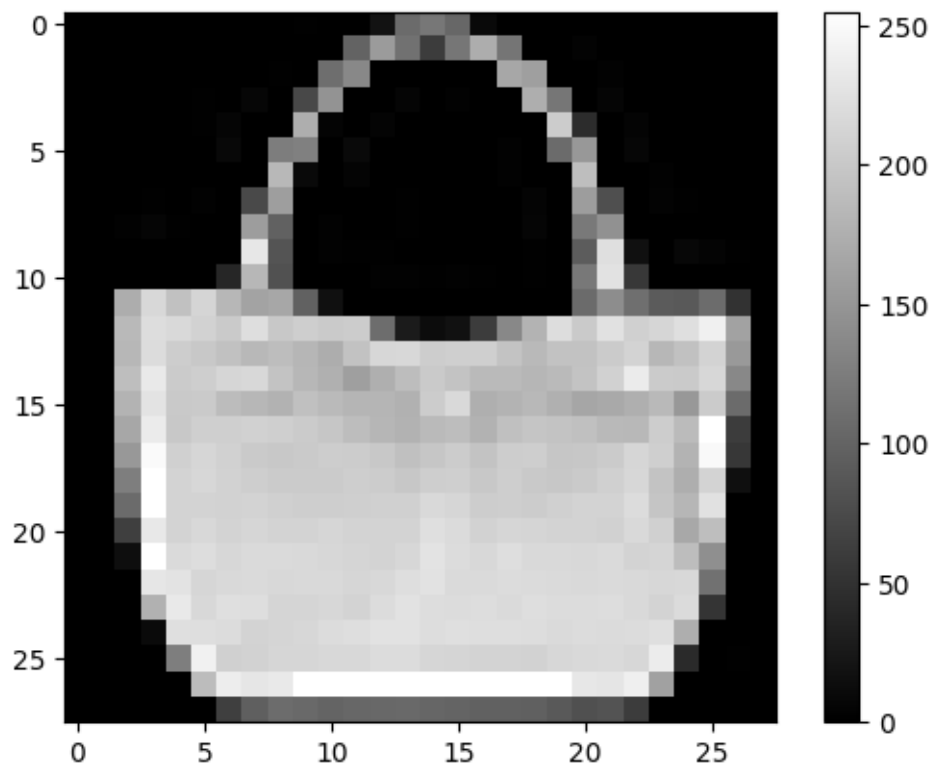
```
print('Print the shapes of images and labels confirm everything is splited as the expected')
print("Training images shape:", train_images.shape)
print("Testing images shape:", test_images.shape)
print("Training labels shape:", train_labels.shape)
print("Testing labels shape:", test_labels.shape)
```

Print the shapes of images and labels confirm everything is splited as the expected

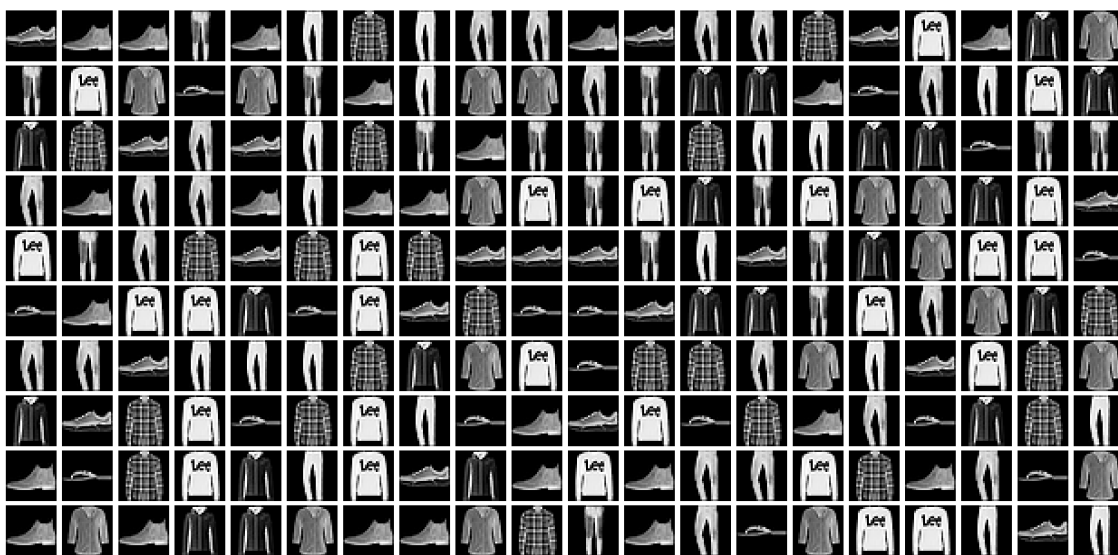
```
Training images shape: (60000, 28, 28)
Testing images shape: (10000, 28, 28)
Training labels shape: (60000,)
Testing labels shape: (10000,)
```

The train image dataset has 60000 images of size 28X28 and the test image dataset has 10000 images of size 28X28

Randomly selected image for visibility
The label of the image: 8



View of Images in the dataset



1.3) Normalize Data Scale pixel values from **0–255** to the range **0–1** for faster and more stable training.

Example:

- Before: a pixel value could be 200
- After: $200 / 255 = 0.78$

```
train_images = train_images/255.0
test_images = test_images/255.0
```

1.4) Reshape Data Reshape images to **$28 \times 28 \times 1$** by adding a channel dimension, making them compatible with CNN layers.

Example:

- Before: (60000, 28, 28) $\rightarrow$ 60,000 grayscale images without channel info
- After: (60000, 28, 28, 1) $\rightarrow$ channel dimension (1) added for CNN input

```
Training images shape after reshaping: (60000, 28, 28, 1)
Training labels shape after reshaping: (10000, 28, 28, 1)
```

1.5) One-Hot Encode Labels Convert class labels (0–9) into **one-hot encoded vectors** with 10 output classes, required for multi-class classification.

Example:

- Original label: 3
- One-hot encoded: [0, 0, 0, 1, 0, 0, 0, 0, 0, 0]

1.6) Verify Dataset Shapes Print the shapes of images and labels after preprocessing to confirm everything is in the expected format.

```
Training images shape: (60000, 28, 28, 1)
Testing images shape: (10000, 28, 28, 1)
Training labels shape: (60000, 10)
Testing labels shape: (10000, 10)
```

0.0.1 2. Model Building

2.1) Basic ANN Model

- **Flatten ($28 \times 28 \rightarrow 784$)** $\rightarrow$ 0 params
- **Dense (128, ReLU)** $\rightarrow$ 100,480 params
- **Dense (64, ReLU)** $\rightarrow$ 8,256 params
- **Dense (10, Softmax)** $\rightarrow$ 650 params

Total params: ~109K

The model summary of ANN model

```

/usr/local/lib/python3.12/dist-
packages/keras/src/layers/reshaping/flatten.py:37: UserWarning: Do not pass an
`input_shape`/`input_dim` argument to a layer. When using Sequential models,
prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(**kwargs)

```

Model: "sequential"

Layer (type)	Output Shape	Param #
flatten (Flatten)	(None, 784)	0
dense (Dense)	(None, 128)	100,480
dense_1 (Dense)	(None, 64)	8,256
dense_2 (Dense)	(None, 10)	650

Total params: 109,386 (427.29 KB)

Trainable params: 109,386 (427.29 KB)

Non-trainable params: 0 (0.00 B)

2.2) Basic CNN Model

- Conv2D (32, 3×3) + MaxPool → 320 params
- Conv2D (64, 3×3) + MaxPool → 18,496 params
- Flatten
- Dense (64, ReLU) → 102,464 params
- Dense (10, Softmax) → 650 params

Total params: ~121K

The model summary of Basic CNN model

```

/usr/local/lib/python3.12/dist-
packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not
pass an `input_shape`/`input_dim` argument to a layer. When using Sequential
models, prefer using an `Input(shape)` object as the first layer in the model

```

instead.

```
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_1 (Conv2D)	(None, 11, 11, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 64)	0
flatten_1 (Flatten)	(None, 1600)	0
dense_3 (Dense)	(None, 64)	102,464
dense_4 (Dense)	(None, 10)	650

Total params: 121,930 (476.29 KB)

Trainable params: 121,930 (476.29 KB)

Non-trainable params: 0 (0.00 B)

2.3) Deeper CNN Model

- Conv2D (32, 3×3) + BN + MaxPool + Dropout → 320 + 64 params
- Conv2D (64, 3×3) + BN + MaxPool + Dropout → 18,496 + 128 params
- Conv2D (128, 3×3) + BN + MaxPool + Dropout → 73,856 + 256 params
- Flatten
- Dense (128, ReLU) + BN + Dropout → 147,584 + 256 params
- Dense (10, Softmax) → 1,290 params

Total params: ~242K

The model summary of Deeper CNN model

Model: "sequential_2"

Layer (type)	Output Shape	Param #
conv2d_2 (Conv2D)	(None, 26, 26, 32)	320
batch_normalization (BatchNormalization)	(None, 26, 26, 32)	128
max_pooling2d_2 (MaxPooling2D)	(None, 13, 13, 32)	0
dropout (Dropout)	(None, 13, 13, 32)	0
conv2d_3 (Conv2D)	(None, 11, 11, 64)	18,496
batch_normalization_1 (BatchNormalization)	(None, 11, 11, 64)	256
max_pooling2d_3 (MaxPooling2D)	(None, 5, 5, 64)	0
dropout_1 (Dropout)	(None, 5, 5, 64)	0
conv2d_4 (Conv2D)	(None, 3, 3, 128)	73,856
batch_normalization_2 (BatchNormalization)	(None, 3, 3, 128)	512
max_pooling2d_4 (MaxPooling2D)	(None, 1, 1, 128)	0
dropout_2 (Dropout)	(None, 1, 1, 128)	0
flatten_2 (Flatten)	(None, 128)	0
dense_5 (Dense)	(None, 128)	16,512
batch_normalization_3 (BatchNormalization)	(None, 128)	512
dropout_3 (Dropout)	(None, 128)	0
dense_6 (Dense)	(None, 10)	1,290

Total params: 111,882 (437.04 KB)

Trainable params: 111,178 (434.29 KB)

Non-trainable params: 704 (2.75 KB)

0.0.2 3. Train models

0.0.3 3. Train models

- **EarlyStopping:** monitors validation loss, stops if no improvement for 5 epochs, restores best weights.
- **ModelCheckpoint:** saves best ANN weights (`best_ann_model_weights.weights.h5`) based on validation loss.
- **Training:**
 - Epochs: up to 30 (early stopping controls actual run)
 - Batch size: 64
 - Validation on test set
 - Callbacks: EarlyStopping + ModelCheckpoint

Training the ANN model

Epoch 1/30

938/938 0s 6ms/step -

accuracy: 0.7695 - loss: 0.6659

Epoch 1: val_loss improved from inf to 0.43125, saving model to

`best_ann_model_weights.weights.h5`

938/938 7s 7ms/step -

accuracy: 0.7695 - loss: 0.6657 - val_accuracy: 0.8499 - val_loss: 0.4313

Epoch 2/30

938/938 0s 5ms/step -

accuracy: 0.8617 - loss: 0.3850

Epoch 2: val_loss improved from 0.43125 to 0.40651, saving model to

`best_ann_model_weights.weights.h5`

938/938 5s 5ms/step -

accuracy: 0.8618 - loss: 0.3850 - val_accuracy: 0.8526 - val_loss: 0.4065

Epoch 3/30

931/938 0s 5ms/step -

accuracy: 0.8745 - loss: 0.3394

Epoch 3: val_loss improved from 0.40651 to 0.38063, saving model to

`best_ann_model_weights.weights.h5`

938/938 6s 6ms/step -

accuracy: 0.8745 - loss: 0.3394 - val_accuracy: 0.8645 - val_loss: 0.3806

Epoch 4/30
927/938 0s 4ms/step -
accuracy: 0.8869 - loss: 0.3086
Epoch 4: val_loss improved from 0.38063 to 0.35256, saving model to
best_ann_model_weights.weights.h5
938/938 5s 5ms/step -
accuracy: 0.8869 - loss: 0.3087 - val_accuracy: 0.8717 - val_loss: 0.3526
Epoch 5/30
934/938 0s 4ms/step -
accuracy: 0.8920 - loss: 0.2895
Epoch 5: val_loss did not improve from 0.35256
938/938 5s 5ms/step -
accuracy: 0.8920 - loss: 0.2895 - val_accuracy: 0.8698 - val_loss: 0.3608
Epoch 6/30
931/938 0s 6ms/step -
accuracy: 0.8964 - loss: 0.2806
Epoch 6: val_loss improved from 0.35256 to 0.34755, saving model to
best_ann_model_weights.weights.h5
938/938 6s 6ms/step -
accuracy: 0.8964 - loss: 0.2806 - val_accuracy: 0.8740 - val_loss: 0.3475
Epoch 7/30
933/938 0s 4ms/step -
accuracy: 0.9006 - loss: 0.2682
Epoch 7: val_loss improved from 0.34755 to 0.34490, saving model to
best_ann_model_weights.weights.h5
938/938 5s 5ms/step -
accuracy: 0.9006 - loss: 0.2682 - val_accuracy: 0.8777 - val_loss: 0.3449
Epoch 8/30
938/938 0s 5ms/step -
accuracy: 0.9053 - loss: 0.2537
Epoch 8: val_loss improved from 0.34490 to 0.32365, saving model to
best_ann_model_weights.weights.h5
938/938 6s 6ms/step -
accuracy: 0.9053 - loss: 0.2537 - val_accuracy: 0.8812 - val_loss: 0.3236
Epoch 9/30
928/938 0s 5ms/step -
accuracy: 0.9078 - loss: 0.2481
Epoch 9: val_loss did not improve from 0.32365
938/938 5s 5ms/step -
accuracy: 0.9078 - loss: 0.2480 - val_accuracy: 0.8835 - val_loss: 0.3294
Epoch 10/30
928/938 0s 4ms/step -
accuracy: 0.9116 - loss: 0.2400
Epoch 10: val_loss improved from 0.32365 to 0.32239, saving model to
best_ann_model_weights.weights.h5
938/938 4s 5ms/step -
accuracy: 0.9116 - loss: 0.2400 - val_accuracy: 0.8876 - val_loss: 0.3224
Epoch 11/30

```

936/938          0s 6ms/step -
accuracy: 0.9154 - loss: 0.2244
Epoch 11: val_loss did not improve from 0.32239
938/938          6s 6ms/step -
accuracy: 0.9154 - loss: 0.2244 - val_accuracy: 0.8777 - val_loss: 0.3487
Epoch 12/30
929/938          0s 4ms/step -
accuracy: 0.9152 - loss: 0.2231
Epoch 12: val_loss did not improve from 0.32239
938/938          4s 5ms/step -
accuracy: 0.9152 - loss: 0.2231 - val_accuracy: 0.8900 - val_loss: 0.3298
Epoch 13/30
938/938          0s 5ms/step -
accuracy: 0.9204 - loss: 0.2103
Epoch 13: val_loss did not improve from 0.32239
938/938          5s 6ms/step -
accuracy: 0.9204 - loss: 0.2103 - val_accuracy: 0.8865 - val_loss: 0.3354
Epoch 14/30
930/938          0s 4ms/step -
accuracy: 0.9226 - loss: 0.2032
Epoch 14: val_loss did not improve from 0.32239
938/938          9s 5ms/step -
accuracy: 0.9226 - loss: 0.2033 - val_accuracy: 0.8854 - val_loss: 0.3424
Epoch 15/30
929/938          0s 7ms/step -
accuracy: 0.9232 - loss: 0.2031
Epoch 15: val_loss did not improve from 0.32239
938/938          7s 8ms/step -
accuracy: 0.9232 - loss: 0.2031 - val_accuracy: 0.8880 - val_loss: 0.3299

```

ANN model training finished.

3.2) Training Basic CNN Model

- **ModelCheckpoint:** saves best Basic CNN weights (`best_basic_cnn_model_weights.weights.h5`) based on validation loss.
- **Training:**
 - Epochs: up to 30 (early stopping controls actual run)
 - Batch size: 64
 - Validation on test set
 - Callbacks: EarlyStopping + ModelCheckpoint

Training the Basic CNN model

Epoch 1/30
937/938 0s 53ms/step -
accuracy: 0.7412 - loss: 0.7293
Epoch 1: val_loss improved from inf to 0.37595, saving model to
best_basic_cnn_model_weights.weights.h5
938/938 54s 56ms/step -
accuracy: 0.7414 - loss: 0.7289 - val_accuracy: 0.8667 - val_loss: 0.3759
Epoch 2/30
937/938 0s 59ms/step -
accuracy: 0.8772 - loss: 0.3450
Epoch 2: val_loss improved from 0.37595 to 0.33663, saving model to
best_basic_cnn_model_weights.weights.h5
938/938 60s 64ms/step -
accuracy: 0.8772 - loss: 0.3450 - val_accuracy: 0.8775 - val_loss: 0.3366
Epoch 3/30
938/938 0s 51ms/step -
accuracy: 0.8905 - loss: 0.3006
Epoch 3: val_loss improved from 0.33663 to 0.30632, saving model to
best_basic_cnn_model_weights.weights.h5
938/938 51s 54ms/step -
accuracy: 0.8905 - loss: 0.3006 - val_accuracy: 0.8877 - val_loss: 0.3063
Epoch 4/30
937/938 0s 50ms/step -
accuracy: 0.9054 - loss: 0.2587
Epoch 4: val_loss improved from 0.30632 to 0.28516, saving model to
best_basic_cnn_model_weights.weights.h5
938/938 50s 53ms/step -
accuracy: 0.9054 - loss: 0.2587 - val_accuracy: 0.8977 - val_loss: 0.2852
Epoch 5/30
938/938 0s 51ms/step -
accuracy: 0.9128 - loss: 0.2382
Epoch 5: val_loss improved from 0.28516 to 0.27563, saving model to
best_basic_cnn_model_weights.weights.h5
938/938 51s 54ms/step -
accuracy: 0.9128 - loss: 0.2382 - val_accuracy: 0.9009 - val_loss: 0.2756
Epoch 6/30
938/938 0s 53ms/step -
accuracy: 0.9221 - loss: 0.2143
Epoch 6: val_loss did not improve from 0.27563
938/938 52s 55ms/step -
accuracy: 0.9221 - loss: 0.2143 - val_accuracy: 0.8978 - val_loss: 0.2825
Epoch 7/30
937/938 0s 52ms/step -
accuracy: 0.9282 - loss: 0.1964
Epoch 7: val_loss improved from 0.27563 to 0.26125, saving model to
best_basic_cnn_model_weights.weights.h5
938/938 82s 55ms/step -
accuracy: 0.9282 - loss: 0.1964 - val_accuracy: 0.9035 - val_loss: 0.2612

Epoch 8/30
937/938 0s 52ms/step -
accuracy: 0.9345 - loss: 0.1777
Epoch 8: val_loss improved from 0.26125 to 0.25891, saving model to
best_basic_cnn_model_weights.weights.h5
938/938 51s 54ms/step -
accuracy: 0.9345 - loss: 0.1777 - val_accuracy: 0.9076 - val_loss: 0.2589
Epoch 9/30
937/938 0s 52ms/step -
accuracy: 0.9406 - loss: 0.1604
Epoch 9: val_loss improved from 0.25891 to 0.25718, saving model to
best_basic_cnn_model_weights.weights.h5
938/938 82s 55ms/step -
accuracy: 0.9406 - loss: 0.1604 - val_accuracy: 0.9083 - val_loss: 0.2572
Epoch 10/30
937/938 0s 52ms/step -
accuracy: 0.9461 - loss: 0.1490
Epoch 10: val_loss improved from 0.25718 to 0.25653, saving model to
best_basic_cnn_model_weights.weights.h5
938/938 51s 55ms/step -
accuracy: 0.9461 - loss: 0.1491 - val_accuracy: 0.9106 - val_loss: 0.2565
Epoch 11/30
937/938 0s 51ms/step -
accuracy: 0.9516 - loss: 0.1317
Epoch 11: val_loss did not improve from 0.25653
938/938 82s 55ms/step -
accuracy: 0.9516 - loss: 0.1317 - val_accuracy: 0.9111 - val_loss: 0.2613
Epoch 12/30
938/938 0s 50ms/step -
accuracy: 0.9556 - loss: 0.1231
Epoch 12: val_loss did not improve from 0.25653
938/938 50s 54ms/step -
accuracy: 0.9556 - loss: 0.1231 - val_accuracy: 0.9128 - val_loss: 0.2607
Epoch 13/30
938/938 0s 51ms/step -
accuracy: 0.9585 - loss: 0.1109
Epoch 13: val_loss did not improve from 0.25653
938/938 53s 56ms/step -
accuracy: 0.9585 - loss: 0.1109 - val_accuracy: 0.9072 - val_loss: 0.2878
Epoch 14/30
937/938 0s 52ms/step -
accuracy: 0.9608 - loss: 0.1062
Epoch 14: val_loss did not improve from 0.25653
938/938 51s 54ms/step -
accuracy: 0.9608 - loss: 0.1062 - val_accuracy: 0.9110 - val_loss: 0.2797
Epoch 15/30
938/938 0s 51ms/step -
accuracy: 0.9641 - loss: 0.0967

Epoch 15: val_loss did not improve from 0.25653
938/938 82s 54ms/step -
accuracy: 0.9641 - loss: 0.0967 - val_accuracy: 0.9112 - val_loss: 0.2968

Basic CNN model training finished.

3.3) Training Deeper CNN Model

- **ModelCheckpoint:** saves best Deeper CNN weights
(best_deeper_cnn_model_weights.weights.h5) based on validation loss.
- **Training:**
 - Epochs: up to 30 (early stopping controls actual run)
 - Batch size: 64
 - Validation on test set
 - Callbacks: EarlyStopping + ModelCheckpoint

Training the Deeper CNN model

Epoch 1/30
938/938 0s 92ms/step -
accuracy: 0.6850 - loss: 0.8948
Epoch 1: val_loss improved from inf to 0.42635, saving model to
best_deeper_cnn_model_weights.weights.h5
938/938 95s 96ms/step -
accuracy: 0.6851 - loss: 0.8946 - val_accuracy: 0.8448 - val_loss: 0.4263
Epoch 2/30
937/938 0s 93ms/step -
accuracy: 0.8283 - loss: 0.4713
Epoch 2: val_loss improved from 0.42635 to 0.38699, saving model to
best_deeper_cnn_model_weights.weights.h5
938/938 141s 96ms/step -
accuracy: 0.8283 - loss: 0.4712 - val_accuracy: 0.8580 - val_loss: 0.3870
Epoch 3/30
937/938 0s 93ms/step -
accuracy: 0.8486 - loss: 0.4143
Epoch 3: val_loss improved from 0.38699 to 0.36387, saving model to
best_deeper_cnn_model_weights.weights.h5
938/938 90s 96ms/step -
accuracy: 0.8486 - loss: 0.4143 - val_accuracy: 0.8660 - val_loss: 0.3639
Epoch 4/30
937/938 0s 94ms/step -
accuracy: 0.8626 - loss: 0.3748
Epoch 4: val_loss improved from 0.36387 to 0.34873, saving model to
best_deeper_cnn_model_weights.weights.h5

```

938/938          91s 97ms/step -
accuracy: 0.8626 - loss: 0.3748 - val_accuracy: 0.8719 - val_loss: 0.3487
Epoch 5/30
937/938          0s 93ms/step -
accuracy: 0.8662 - loss: 0.3608
Epoch 5: val_loss improved from 0.34873 to 0.32747, saving model to
best_deeper_cnn_model_weights.weights.h5
938/938          141s 97ms/step -
accuracy: 0.8662 - loss: 0.3608 - val_accuracy: 0.8775 - val_loss: 0.3275
Epoch 6/30
937/938          0s 94ms/step -
accuracy: 0.8747 - loss: 0.3446
Epoch 6: val_loss did not improve from 0.32747
938/938          91s 97ms/step -
accuracy: 0.8747 - loss: 0.3446 - val_accuracy: 0.8725 - val_loss: 0.3396
Epoch 7/30
937/938          0s 93ms/step -
accuracy: 0.8787 - loss: 0.3310
Epoch 7: val_loss improved from 0.32747 to 0.29744, saving model to
best_deeper_cnn_model_weights.weights.h5
938/938          142s 97ms/step -
accuracy: 0.8787 - loss: 0.3310 - val_accuracy: 0.8908 - val_loss: 0.2974
Epoch 8/30
938/938          0s 94ms/step -
accuracy: 0.8834 - loss: 0.3161
Epoch 8: val_loss did not improve from 0.29744
938/938          91s 97ms/step -
accuracy: 0.8834 - loss: 0.3161 - val_accuracy: 0.8855 - val_loss: 0.3056
Epoch 9/30
938/938          0s 93ms/step -
accuracy: 0.8839 - loss: 0.3140
Epoch 9: val_loss did not improve from 0.29744
938/938          90s 96ms/step -
accuracy: 0.8839 - loss: 0.3140 - val_accuracy: 0.8900 - val_loss: 0.3019
Epoch 10/30
937/938          0s 93ms/step -
accuracy: 0.8878 - loss: 0.3031
Epoch 10: val_loss did not improve from 0.29744
938/938          90s 96ms/step -
accuracy: 0.8878 - loss: 0.3031 - val_accuracy: 0.8826 - val_loss: 0.3176
Epoch 11/30
937/938          0s 93ms/step -
accuracy: 0.8901 - loss: 0.2949
Epoch 11: val_loss improved from 0.29744 to 0.29666, saving model to
best_deeper_cnn_model_weights.weights.h5
938/938          142s 96ms/step -
accuracy: 0.8901 - loss: 0.2949 - val_accuracy: 0.8889 - val_loss: 0.2967
Epoch 12/30

```

```

938/938          0s 94ms/step -
accuracy: 0.8941 - loss: 0.2870
Epoch 12: val_loss improved from 0.29666 to 0.28824, saving model to
best_deeper_cnn_model_weights.weights.h5
938/938          143s 97ms/step -
accuracy: 0.8941 - loss: 0.2870 - val_accuracy: 0.8945 - val_loss: 0.2882
Epoch 13/30
937/938          0s 94ms/step -
accuracy: 0.8957 - loss: 0.2805
Epoch 13: val_loss did not improve from 0.28824
938/938          91s 97ms/step -
accuracy: 0.8957 - loss: 0.2805 - val_accuracy: 0.8867 - val_loss: 0.3015
Epoch 14/30
938/938          0s 92ms/step -
accuracy: 0.8941 - loss: 0.2791
Epoch 14: val_loss improved from 0.28824 to 0.27953, saving model to
best_deeper_cnn_model_weights.weights.h5
938/938          141s 96ms/step -
accuracy: 0.8941 - loss: 0.2791 - val_accuracy: 0.8977 - val_loss: 0.2795
Epoch 15/30
938/938          0s 92ms/step -
accuracy: 0.8989 - loss: 0.2732
Epoch 15: val_loss did not improve from 0.27953
938/938          90s 96ms/step -
accuracy: 0.8989 - loss: 0.2733 - val_accuracy: 0.8913 - val_loss: 0.2913
Epoch 16/30
938/938          0s 93ms/step -
accuracy: 0.8997 - loss: 0.2741
Epoch 16: val_loss did not improve from 0.27953
938/938          92s 98ms/step -
accuracy: 0.8997 - loss: 0.2741 - val_accuracy: 0.8914 - val_loss: 0.2918
Epoch 17/30
937/938          0s 93ms/step -
accuracy: 0.9004 - loss: 0.2702
Epoch 17: val_loss did not improve from 0.27953
938/938          90s 96ms/step -
accuracy: 0.9004 - loss: 0.2702 - val_accuracy: 0.8938 - val_loss: 0.2850
Epoch 18/30
937/938          0s 93ms/step -
accuracy: 0.9001 - loss: 0.2683
Epoch 18: val_loss did not improve from 0.27953
938/938          90s 96ms/step -
accuracy: 0.9001 - loss: 0.2683 - val_accuracy: 0.8923 - val_loss: 0.2880
Epoch 19/30
938/938          0s 92ms/step -
accuracy: 0.9030 - loss: 0.2603
Epoch 19: val_loss did not improve from 0.27953
938/938          142s 96ms/step -

```

accuracy: 0.9030 - loss: 0.2603 - val_accuracy: 0.8954 - val_loss: 0.2874

Deeper CNN model training finished.

0.0.4 4. Model Evaluation

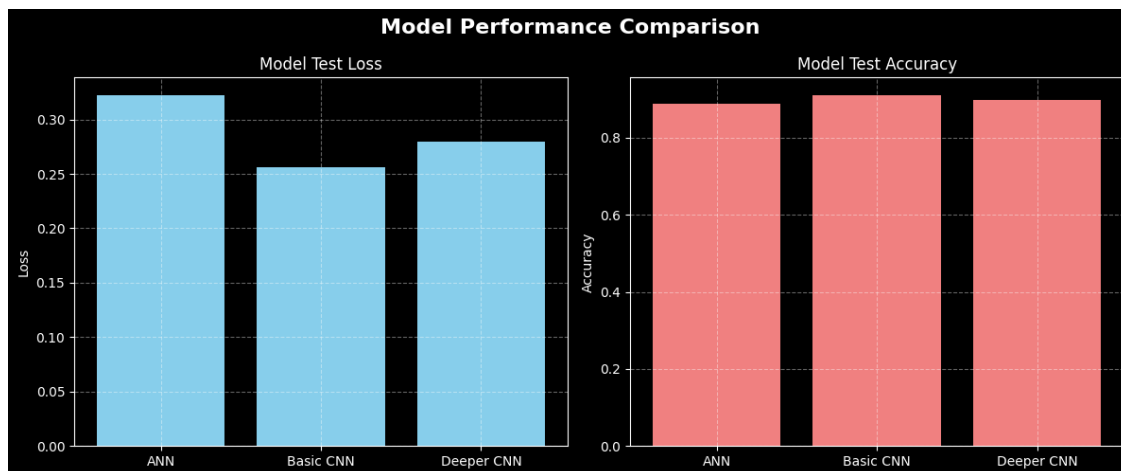
4.1) Model Evaluation Code Explanation Import utilities → Load weights (ANN, Basic CNN, Deeper CNN) → Evaluate on test set → Get loss & accuracy → Print results

The accuracy of the three model are as follows

ANN Accuracy: 88.76%

Basic CNN Accuracy: 91.06%

Deeper CNN Accuracy: 89.77%

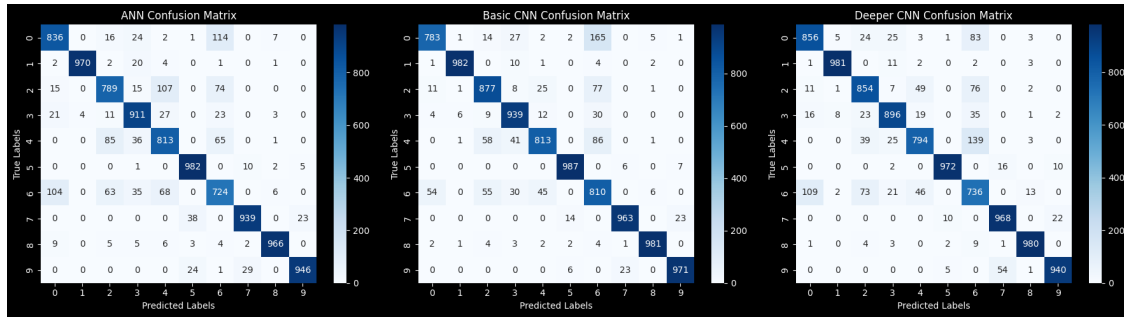


Interpretation : The Basis CNN has out perform and has better prediction accuracy compared to the ANN and Deeper CNN model. Although the model run for similar number of epoch before callback (ANN 15 epoch, Basic_CNN 15 epoch, Deeper_CNN 19 epoch)

4.2) Model Evaluation (Confusion Matrix & Report)

- Predict test images (ANN, Basic CNN, Deeper CNN)
- Convert one-hot labels to integers
- Get predicted labels with `argmax`
- Print confusion matrix & classification report for each model

```
313/313          1s 2ms/step
313/313          4s 8ms/step
313/313          5s 15ms/step
```



4.3) Model Comparison

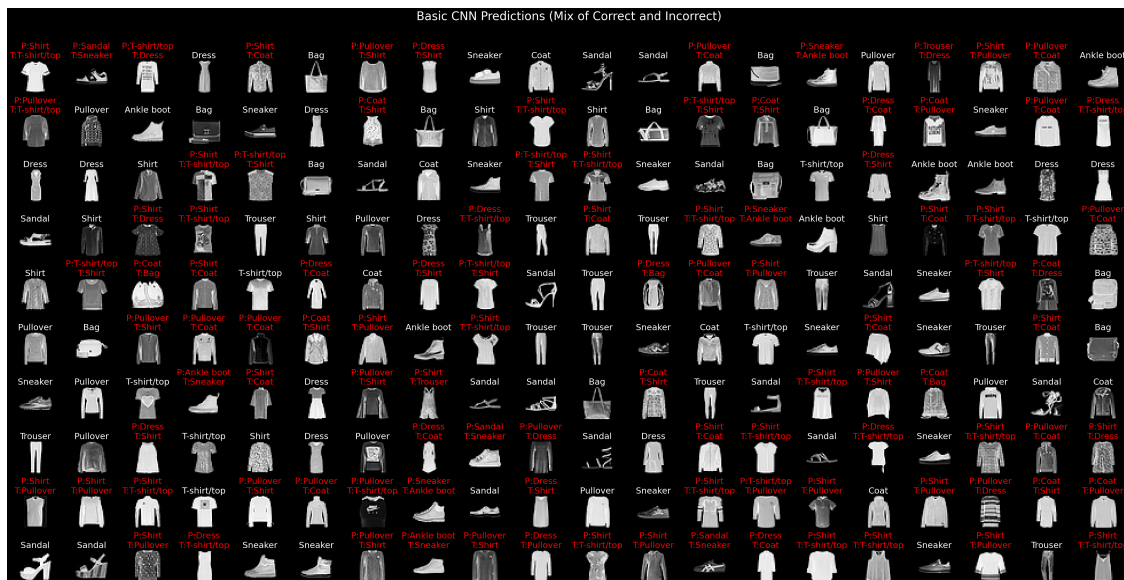
- Created a dataframe to store **test loss** and **test accuracy** for ANN, Basic CNN, and Deeper CNN.
- Plotted training history for each model (accuracy & loss vs. epochs).
- Helps compare how models learn and generalize over time.

4.4) Predictions This code: - Uses the trained **Basic CNN model** to predict on test images.

- Identifies **correctly** and **incorrectly** classified samples.
- Randomly selects 100 correct and 100 incorrect predictions.
- Displays them in a **20×10 grid**:
- Correct predictions → **Green title** (only predicted label).
- Incorrect predictions → **Red title** (Predicted vs True label).

313/313

3s 8ms/step



0.1 Project Summary and Conclusion

This project successfully implemented and compared three different deep learning models (ANN, Basic CNN, and Deeper CNN) for classifying images from the Fashion-MNIST dataset.

```
[NbConvertApp] Converting notebook /content/drive/My Drive/Colab
Notebooks/Classification_of_Fashion_accessories.ipynb to pdf
/usr/local/share/jupyter/nbconvert/templates/latex/display_priority.j2:32:
UserWarning: Your element with mimetype(s) dict_keys(['text/html']) is not able
to be represented.
  ((*- endblock -*))
[WARNING] Could not parse YAML metadata at line 5 column 1: not an object
[NbConvertApp] Support files will be in
Classification_of_Fashion_accessories_files/
[NbConvertApp] Making directory ./Classification_of_Fashion_accessories_files
[NbConvertApp] Writing 73603 bytes to notebook.tex
[NbConvertApp] Building PDF
[NbConvertApp] Running xelatex 3 times: ['xelatex', 'notebook.tex', '-quiet']
[NbConvertApp] Running bibtex 1 time: ['bibtex', 'notebook']
[NbConvertApp] WARNING | bibtex had problems, most likely because there were no
citations
[NbConvertApp] PDF successfully created
[NbConvertApp] Writing 916668 bytes to /content/drive/My Drive/Colab
Notebooks/Classification_of_Fashion_accessories.pdf
```